

```
SecFilter "\.\/"
SecFilterSelective SCRIPT_FILENAME "/scripts/foo.cgi" chain
SecFilterSelective ARG_home "!^[a-zA-Z]{15,}\\.txt"
```

The last two filters are chained, and will reject all parameters to the “home” argument that is a filename of more than 15 alpha characters, and that doesn’t have a “.txt” extension.

Similarly, you can prevent predictable resource location attacks also, and protect against sensitive file misuse, with two recommended solutions. First, remove files that are not intended for public viewing from all Web-server-accessible directories. After this, you can create security filters to identify if someone probes for these files:

```
SecFilterSelective REQUEST_URI "^/(scripts|cgi-local|htbin|cgibin|cgis|win-cgi|cgi-win|bin)/"
SecFilterSelective REQUEST_URI ".*\\.(bak|old|orig|backup|c)$"
```

These two filters will deny access to both—unused, but commonly scanned for directories, and files with common backup extensions.

Web pages that are dynamically created by the directory-indexing function will have a title that starts with “Index of/”. We can use this as a signature, and add the following directives to catch and deny access to this data:

```
SecFilterScanOutput On
SecFilterSelective OUTPUT "\\<title>Index of /"
```

Information leakage

Here, we are introduced to the *OUTPUT* filtering capabilities of *mod_security*, which you should enable by adding *SecFilterScanOutput On* in the configuration file. We can easily set up a filter to watch for common database error messages being sent to the client, and then generate a generic 500 status code instead of the verbose message:

```
SecFilterScanOutput On
SecFilterSelective OUTPUT "An Error Has Occurred" status:500
```

```
SecFilterSelective OUTPUT "Fatal error:"
```

Output filtering can also be used to detect successful intrusions. These rules will monitor output, and detect typical keywords resulting from a command execution on the server.

```
SecFilterSelective OUTPUT "Volume Serial Number"
SecFilterSelective OUTPUT "Command completed"
SecFilterSelective OUTPUT "Bad command or filename"
SecFilterSelective OUTPUT "file(s) copied"
SecFilterSelective OUTPUT "Index of /cgi-bin/"
SecFilterSelective OUTPUT ".*uidv=\\(\\"
```

Secure file uploads

Mod_security is capable of intercepting files uploaded through POST requests and multi-part/form-data encoding through PUT requests. It will always upload files to a temporary directory. You can choose the directory using the *SecUploadDir* directive:

```
SecUploadDir /tmp
```

It is better to choose a private directory for file storage, somewhere that only the Web-server user account is allowed access. Otherwise, other server users may be able to access files uploaded through the Web server. You can choose to execute an external script to verify a file before it is allowed to go through to the application. The *SecUploadApproveScript* directive enables this, like the following example:

```
SecUploadApproveScript /usr/local/apache/bin/upload_verify.pl
```

RFI attacks

RFI attacks are generally easy to detect, with something like the following directive:

```
SecRule ARGS "@rx (?i)(f|ht)tps?://([\\^/])" msg:'Remote File Inclusion attack'
# To detect inclusions containing IP address
SecRule ARGS "@rx (ht|f)tps?://([01]?d{0,4})d{25[0-5]}\\."
```

TechnoMail - Enterprise Email Server
Anti SPAM, Anti Virus, Email Content Filtering

Firewall, Internet Access Control
Content Filtering, Site Blocking

Bandwidth Management System

Managed Email Hosting Solutions

TechnoInfotech®

1, Vikas Permisses, 11 Bank Street, Fort, Mumbai, India - 400 001. Mobile: 09167399917. info@technoinfotech.com